Replication Study of Web-Based PLC Malware: Firmware Analysis, Service Worker Resurrection, and Behavioral Anomaly Detection

Hojune Kim
School of Computing
Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea
hojunekim@kaist.ac.kr

***Abstract*—This is a replication study of "Compromising Industrial Processes using Web-Based PLC Malware" (Pickern et al., NDSS 2024), which proposed IronSpider — a web-based malware that persists within the web interface of modern programmable logic controllers (PLCs) and bypasses factory reset via a Service Worker resurrection scheme. This study reproduces the paper's firmware complexity analysis (E1) across four WAGO 750-8XXX firmware versions ranging from 2019 to 2023, conducts additional qualitative analysis on firmware JavaScript API features, and created a standalone browser testbed for the full six-step resurrection cycle (E4) including survival of hardware replacement. Furthermore, this study implements the three key attack capabilities inspired by IronSpider — sensor spoofing, alarm supression, and actuator manipulation — by manual malware injection, which is analogous to CVE-driven code injection in the paper. Finally, this study strives to solve the paper's open detection problem of web-based PLC malware by implementing a server-side runtime detector inside OpenPLC's Flask webserver. Tier 1 alert shows zero false positive over five minutes of normal operation by setting up detection logic for two browser-generated request headers: `Sec-Fetch-Dest: empty` and a `Referer` originating from a `.js` context. The detection technique is evaluated with three evasion methods, two of which defeat Tier 1 with suggestions on countermeasures.**

***Index Terms*—PLC malware, industrial control systems, Service Worker, web security, behavioral detection, ICS**

I. Introduction

Industrial control systems (ICSs) are the backbone of critical infrastructures such as water treatment plant, power grids, and manufacturing factories. They have been considered to be free of web-based attack vectors, due to its traditional isolation from the open Internet. However, this assumption has become obsolete as modern PLCs gradually provide more support for browser-based human-machine interfaces (HMIs) and embedded web server, which allow for remote system monitoring and configuration. The boundary between operational technology (OT) and information technology (IT) is fading, and ICSs are now open to web-based attack.

Pickren et al. [1] proposed IronSpider, a web-based PLC malware (WB PLC malware) that hinders the system entirely within the operator's browser. Unlike traditional PLC malware (e.g., Stuxnet [2], HARVEY []), IronSpider does not require prior knowledge of firmware or kernel exploits; it only utilises browser APIs to control the system as if it were a legitimate human operator. The paper demonstrates key attack capabilities such as sensor spoofing and actuator manipulation, along with a Service Worker (SW) resurrection mechanism that revives the malware after factory reset, surviving even hardware replacement. Another key finding is that four established JavaScript malware detectors (Cujo, Zozzle, JaSt, JStap) all failed to categorise IronSpider as malicious, since the malware does not contain any aggressive or syntactically suspicious code patterns [1].

This study contains a structured replication study conducted over 19 days with four contributions:

1) **Firmware growth analysis (E1).** This study replicates the paper's SLOC and cyclomatic complexity measurements on four WAGO firmware versions (two from the paper, two intermediate chosen by this study). Also, we extend the analysis qualitatively by observing any change in JavaScript API surface between versions.
2) **Service Worker resurrection testbed (E4).** This study builds a standalone Node.js/HTTPS testbed showing the full six-step resurrection process of WB PLC malware and demonstrate hardware replacement survival and the 24-hour cache limit of SWs.
3) **OpenPLC capability demonstration.** This study implements the three key attack capabilities of IronSpider on OpenPLC via manual payload injection — an adaptation which preserves the attack characteristics while acknowledging that CVE-driven code injection in OpenPLC is not feasible, unlike in the original WAGO case.
4) **Behavioral anomaly detection.** This study implements a server-side detector as a component of OpenPLC's Flask webserver, addressing the detection gap stated in the paper. The Tier 1 heuristic makes use of browser-generated Fetch metadata headers to detect SW constantly checking for the malware's presence, while keeping the number of false positives to zero.

The remainder of the paper is organised as follows. Section II covers background on ICS architecture, WB PLC malware, and the OpenPLC browser APIs exploited. Section III describes the test environment and experimental adaptations. Sections IV–VII present results. Section VIII discusses implications and limitations. Section IX concludes.

II. Background

A. ICS Architecture and the PERA Model

Industrial control systems typically adopts the Purdue Enterprise Reference Architecture (PERA) model, which categorises assets into five levels: Level 0 (field devices and sensors), Level 1 (basic control including PLCs, RTUs), Level 2 (supervisory control including HMIs, SCADA), Level 3 (site operations network), and Level 4 (enterprise network). WB PLC malware places itself entirely within the OT boundary (Level 2), spoofing HMI values and controlling actuators on its own.

B. Web-Based PLC Malware vs. Traditional PLC Malware

Traditional PLC malware (e.g., Stuxnet [2], Harvey [3]) often requires specific knowledge about the victim's instruction set and PLC firmware. In comparison, WB PLC malware only exploits the victim browser's web API for controlling the underlying PLC, imitating the legitimate human operator behind the HMI.

C. Service Workers and the Resurrection Mechanism

A Service Worker (SW) is a JavaScript file that the browser registers as a persistent background proxy for a web origin [4]. Key properties that IronSpider exploits:

- **Persistence.** SW registrations are stored on the browser's side; deleting the injected SW file from the server does not remove the registered SW.
- **CacheStorage.** SWs can store data in `CacheStorage`, which persists without explicit user action or storage pressure.
- **Same-origin fetch privilege.** A SW registered on the PLC's web context can call any same-origin web API without cross-origin resourcce sharing (CORS) restriction, exploiting the operator's authenticated session.
- **DOM bypass via `respondWith`.** SWs do not have direct DOM access, but they can intercept HTTP responses and inject arbitrary JavaScript code. The injected code runs in the page context with full DOM access.

The resurrection cycle (Fig. 4 of [1]) works as follows: after initial infection, the SW caches the malware payload in CacheStorage. If a factory reset occurs, the SW detects the 404 after trying to GET the malware. It then retrieves the cached copy, and re-uploads it using the same CVE chain exploited to inject the malware.

D. Static JS Malware Detection and Its Limits

The paper attempted at detecting IronSpider using four static JavaScript analysers: Cujo [6], Zozzle [7], JaSt [8], and JStap [9]. All four classified IronSpider as benign, since it lacks aggressive indicators such as mining cryptocurrency in WebAssembly.

III. Methodology

A. Testbed Overview

The study uses two separate test environments:

1) **E1 firmware analysis.** WAGO 750-8XXX firware images v3.0.39 (May 2019) and v4.02.13 (Mar 2023) are the two versions discussed in the original paper. This study additionally analyses two intermediate versions: v3.1.7 (Jul 2019) and v3.09.04 (Mar 2022). These firmware versions are analysed using the paper's published Docker container, using the SLOC and cyclomatic complexity tools.
2) **E4 resurrection testbed.** This study creates a Node.js HTTPS server, which serves a simulated PLC dashboard. Three routes replicate the attack relevant server functionality: `GET /*` (static file server), `POST /reset` (factory reset: deletes `malware.js` from disk), and `POST /upload` (file write: analogue of CVE-2022-45140).

B. OpenPLC Adaptation

OpenPLC Runtime v3 [5] is an open-source PLC with a Flask-based web interface, which provides a monitor/write API structure. This study installed a water pump ladder logic program (Structured Text) that controls a simulated pump relay (`%QX0.0`) based on a water level sensor (`%IW0`) with configurable high/low setpoints and an alarm output (`%QX0.1`).

Adaptation from paper. The original paper exploits the three-CVE chain to deliver the malware inside the PLC's web directory. OpenPLC's Flask architecture lacks such features; Flask's `static/` folder is the structural equivalent, but this confines SW scope to `/static/` rather than `/`. Accordingly, this study manually injects both the SW registration and IronSpider capabilities directly into `static/` and loading them on OpenPLC's monitoring page template (`pages.py`). This simulates the *effect* of the CVE-based delivery, without replicating the delivery mechanism itself. This marks a clear distinction from the paper's original scenario.

C. Detection Extension

The detector (`ironspider_detector.py`) is a Python program included inside OpenPLC's Flask webserver using a `@before_request` hook. It adopts three detection tiers of increasing false-positive risk:

- **Tier 1 (deterministic):** This monitors browser-generated headers for incoming GET requests. Alerts are generated when 3 or more requests with `Sec-Fetch-Dest: empty` and a `Referer` URL ending in `.js` arrive within a 90-second sliding window.
- **Tier 2 (statistical):** This monitors whether the rate of `/point-write` call exceeded 2 writes/second over a 5-second sliding window;
- **Tier 3 (heuristic):** This monitors the latency between a `/monitor-update` (sensor read) and a subsequent `/point-write` (actuator write); alerts if under 500 ms.

The rationale behind setting up Tier 1 is explained in detail in Section VII. All the parameters for detection tiers can be configured as constants, and alert outputs are printed to the server console. The alerts are also accessible via a `/ironspider-alerts` endpoint.

IV. Firmware Growth Analysis (E1)

A. SLOC and Cyclomatic Complexity

Table I shows the SLOC and cyclomatic complexity measurements for all four WAGO 750-8XXX firmware versions.The two endpoints (v3.0.39 and v4.02.13) show the paper's reported figures that are verified by this study; the two intermediate versions (v3.1.7 and v3.09.04) are the extension.

TABLE I
WAGO WBM codebase growth across four firmware versions.

Version (Date)	Total SLOC	JS/PHP	Complexity
v3.0.39 (May 2019)	13,188	12,868/320	4,529
v3.1.7 (Jul 2019)	13,472	13,150/322	4,574
v3.09.04 (Mar 2022)	36,544	35,994/550	11,379
v4.02.13 (Mar 2023)	39,007	38,444/563	11,974

Total SLOC grew by 196% from v3.0.39 to v4.02.13, which is consistent with the original paper's claim of "over 195%". The intermediate versions show that the growth of web application codebase was not a sudden jump, but rather an incremental process. The table reveals that the major expansion occurred between mid-2019 and early 2022, with SLOC increasing 171% from v3.1.7 to v3.09.04. Cyclomatic complexity follows the same pattern, increasing 164% overall. Figure 1 visualises this growth across the firmware versions.

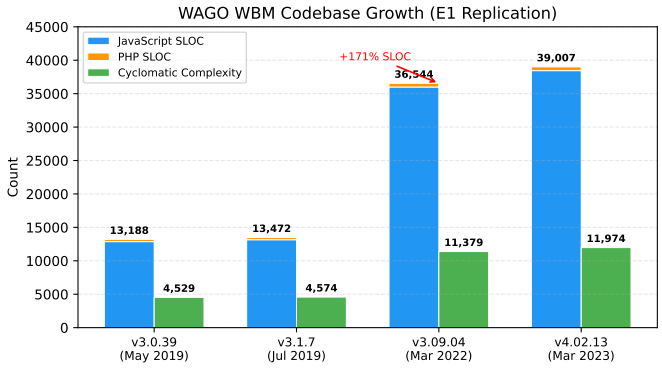

Fig. 1. WAGO WBM codebase growth across four firmware versions, showing total SLOC (stacked JS/PHP) and cyclomatic complexity.

B. Qualitative API Surface Analysis

This study also provides a qualitative analysis on the updated web-based features from v3.0.39 and v4.02.13, revealing *what kind* of functionality was added. Table II summarizes the new functional categories.

The newly added features such as AIDE intrusion detection, firewall configuration, and certificate management are controlled by the smae web interface that the CVE chain can compromise. There is a possiblity of these tools being exploited by an attacker. Meanwhile, Docker on PLCs and persistent MQTT cloud channels may suggest new attack surface that do not exist in the 2019 firmware.

TABLE II
New JavaScript functional categories in v4.02.13.

Category	New files	Notable additions
Core framework	18	Plugin loader (`pfc.js`)
Security/Auth	12	AIDE IDS, firewall, cert mgmt
Industrial proto.	12	OPC UA PKI, WDA scan toggle
Monitoring	12	Pcap logging, runtime task list
Cloud/Remote	7	Azure/AWS/MQTT connectivity
Container/Infra	4	Docker management, pkg server
API documentation	1	OpenAPI self-describing REST

V. Service Worker Resurrection (E4)

A. Six-Step Resurrection Cycle

This study verified all the steps of the malware resurrection cycle stated in Section IV-C of [1]:

1) **SW registration.** When loading the page for the first time, the browser fetches `sw.js` and registers it at scope `/`. Looking at DevTools confirms "activated and is running."
2) **Payload delivery.** As `malware.js` is injected, it starts to execute display spoofing (sensor values turn green) and exfiltration simulation for credentials logged to console.
3) **Factory reset.** `malware.js` from the server, simulating a factor reset. The SW remains registered in the browser profile.
4) **SW detects absence.** The SW's `setInterval` fetches for `/malware.js` within 5 seconds, and receives HTTP 404.
5) **Re-upload.** The SW retrieves the payload from`CacheStorage` and POSTs it to `/upload` (this is analogous to the original paper's CVE chain).Server log confirms: "`RESURRECTION: wrote malware.js`."
6) **Payload re-execution.** The SW sends `postMessage({type: 'RESURRECTED'})` to all open tabs. Display spoofing resumes with the re-injected malware.

All six steps above executed autonomously wihtin one SW polling interval of 5 seconds. This confirms the paper's claim on malware resurrection after the factory reset.

B. Hardware Replacement Survival

This study also evaluated the paper's claim that resurrection survives hardware replacement. This was achieved by simulating full server process replacement:

1) Record server PID via `GET /server-info` (PID 3,725,031).
2) Kill the Node.js process (`Ctrl+C`).

3) Delete `malware.js` and `sw.js` from disk.
4) Restart the server.
5) Record new PID (3,728,686) — different process confirmed.

The browser's SW registration and activation persisted, despite the complete server replacement. The full resurrection cycle was executed within the SW polling interval, re-uploading both the cached `malware.js` *and* `sw.js` to the new server instance. This supports the paper's claim that the SW registration resides in the browser, not the physical device.

C. The 24-Hour Cache Limit

The paper mentions a "24-hour cache limit" as a limitation for SW. This study identified two relevant mechanisms in the W3C SW specification [4] to look into this claim further:

1) **SW script update check interval** (§9.2): the browser fetches the SW script at most once per 24 hours per navigation, and compares it with the registered SW byte-for-byte.
2) **CacheStorage eviction**: the specification does *not* make a 24-hour eviction policy compulsory. Storage entries are evicted only under explicit user action or storage pressure.

This study empirically confirmed that CacheStorage entries persist across browser tab closure, server restarts, and page refreshes. The only reliable solution to eliminate the SW is explicit SW unregistration via DevTools.

VI. OpenPLC Capability Demonstration

This study implemented and confirmed the functionality of the three key IronSpider capabilities depicted in the original paper. This was achieved on OpenPLC Runtime v3 with a custom water pump control program (`water_pump.st`).

A. Capability 1: Sensor Spoofing

The injected malware wraps the response from `/monitor-update` by gaining access to the DOM update in the monitoring page. The `ironspiderSpoof()` function parses the HTML table from the API, finds the row containing `water_level`, records the actual value to `localStorage`, and returns a fabricated reading value before display. The operator observes a normal water level when the actual value may be dangerously high.

B. Capability 2: Alarm Suppression via Spoofing

When the water level exceeds the alarm threshold of 80, the PLC sets `alarm_active` to `TRUE`. The spoofing The spoofing function detects the `bool_true.png` indicator and replaces with `bool_false.png` in the DOM. The operator is not able to see the actual alarm, potentially ignoring a dangerous situation.

C. Capability 3: Actuator Manipulation

The malware runs a `setInterval` loop, calling `/point-write?value=1&address=%QX0.0` every 500 ms. This changes the actual Boolean value for the pump relay, turning it ON regardless of the ladder logic's output. This bypasses the PLC control loop by using the web API.

While testing for this capability, intermittent toggling of`pump_running` between TRUE and FALSE was observed. This is due to the PLC's scan cycle writing the actual, computed value to the same address (`%QX0.0`). In a real deployment, combining actuator manipulation while spoofing the pump relay reading would hide the override from the operator entirely.

VII. Behavioral Anomaly Detection

A. Design Rationale for Tier 1

The paper identifies runtime behavioral detection as an "open problem" because IronSpider uses only legitimate browser APIs with no syntactically suspicious patterns. We observe that while the *payload* is indistinguishable from benign JavaScript, the *delivery mechanism* — a Service Worker polling loop — produces distinguishable HTTP request metadata.

The paper leaves detection of IronSpider as an "open problem" since IronSpider uses only legitimate browser API with no suspicious patterns detected by JavaScript analysers. This study observed that while the *payload* itself is indistinguishable from benign JavaScript, the *SW polling loop* for the malware may produce distinguialbe HTTP request metadata useful for detection.

This study focused on detecting SW-originated fetch using two browser-generated headers:

- **`Sec-Fetch-Dest: empty`.** This header is set for all `fetch()` API calls according to the W3C Fetch Metadata specification, while a `<script src>` tag sets `Sec-Fetch-Dest: script` instead. JavaScript cannot forge or suppress this header.
- **Referer ending in `.js`.**`fetch()` calls initiated by SW sets the SW script URL as Referer (e.g., `/static/sw.js` in OpenPLC's case). Page-originated `fetch()` calls has the page URL as the value for Referer (e.g., `/monitoring`), which does not end in `.js`.

Neither signal alone is sufficient: `Sec-Fetch-Dest: empty` also matches page-originated `fetch()` calls (e.g., AJAX polling in the monitoring page), and a `.js` Referer could theoretically appear in benign SW scenarios. The conjunction of both signals within a sliding window (3 requests in 90 seconds) identifies the characteristic SW polling loop.

1) Iterative Development: The final design was reached through two failed iterations:

1) **Count-based (rejected):** counting all GET requests to `/static/malware.js` produced immediate false positives because the monitoring page loads the file via `<script src>`.

2) **Cache-Control filter (rejected):** the WHATWG Fetch specification states that `cache: 'no-store'` appends `Cache-Control: no-cache` to the request. However, Chromium-based browsers (Chrome, Brave) do *not* send this header — they handle the cache mode internally. The `Cache-Control` header is completely absent from SW `fetch()` requests on these browsers.

A debug header dump revealed `Sec-Fetch-Dest` and `Referer` as the correct discriminators, leading to the final design.

B. False Positive Evaluation

We disabled the malware injection (commented out the payload block in `pages.py`) and removed `malware.js` and `sw.js` from the server. The detector ran for five minutes of normal operator interaction (page loads, monitoring refreshes, manual point writes). **Zero Tier 1 alerts** were generated.

Normal browser requests to OpenPLC produce `Sec-Fetch-Dest: document` (page loads), `Sec-Fetch-Dest: script` (`<script src>` loads), or `Sec-Fetch-Dest: empty` with a non-`.js` Referer (AJAX polling from the monitoring page). None of these combinations match the Tier 1 conjunction.

C. Evasion Evaluation

We tested three evasion techniques against the Tier 1 detector:

TABLE III
EVASION TECHNIQUES AGAINST THE TIER 1 DETECTOR.

Evasion	Technique	Det.?
SW rename	Rename to `resurrect.js`	Yes
Slow polling	Interval $\rightarrow$ 90 s	No
Referer suppress	`no-referrer` policy	No

E1 (SW rename): The detector is filename-agnostic — it checks whether any `Referer` ends in `.js`, not a specific filename. The alert fired correctly after renaming.

E2 (slow polling): With one request per 90 seconds, only one request accumulates per window, never reaching the threshold of 3. *Countermeasure:* reduce the threshold to 1 (any single SW-context fetch triggers an alert), at the cost of potential false positives if a benign SW is deployed on the PLC. Alternatively, extend the window beyond 90 seconds.

E3 (Referer suppression): Adding `referrerPolicy: 'no-referrer'` to the fetch options causes the browser to omit the `Referer` header entirely. With no Referer to inspect, Tier 1's second signal is eliminated. *Countermeasure:* treat the *absence* of a Referer header on a `Sec-Fetch-Dest: empty` request as suspicious, since legitimate page-originated fetch calls always include a Referer in standard browser configurations.

VIII. DISCUSSION

A. Replication Fidelity

Our E1 results reproduce the paper's SLOC and complexity figures within rounding error, validating the measurement methodology. The intermediate firmware versions provide new evidence that the codebase expansion was gradual, driven primarily by the v3.1.7 $\rightarrow$ v3.09.04 transition (2019–2022), which added the plugin architecture, cloud connectivity, and Docker support.

The E4 testbed successfully replicates all six resurrection steps and provides the first empirical confirmation of the hardware replacement survival claim. Our clarification of the "24-hour cache limit" as the SW update check interval (not CacheStorage eviction) resolves an ambiguity in the original paper.

B. Limitations

OpenPLC adaptation. The most significant limitation is the absence of a CVE-based delivery vector on OpenPLC. Our manual injection simulates the *effect* of the exploit chain but does not validate the *delivery* phase. The SW scope is also confined to `/static/` rather than `/`, which limits the SW's ability to intercept all page fetches.

Single browser tested. The detector's Tier 1 signals (`Sec-Fetch-Dest` and `Referer` behavior) were validated on Brave (Chromium-based). Firefox and Safari may set these headers differently; cross-browser validation is left for future work.

False positive test scope. Five minutes of normal operation is a limited evaluation. A production deployment would require multi-day testing across multiple operator workflows and network conditions.

Tier 2 and Tier 3 not rigorously evaluated. We focused evaluation effort on Tier 1 as the primary contribution. Tier 2 (write rate) and Tier 3 (read-write latency) were implemented and observed to fire during attack scenarios but were not subjected to systematic false positive testing.

C. Implications for ICS Defense

The detection gap identified by the paper is real but not insurmountable. Server-side behavioral detection using browser-generated Fetch Metadata headers offers a practical defense layer that requires no client-side instrumentation and no modification to the browser. The approach is deployable as a lightweight WAF module, consistent with the "PLC-configured WAF" countermeasure proposed in Table VII of [1].

The evasion analysis reveals that the Referer header is the weaker of the two signals. A more robust design would combine Tier 1 with Content Security Policy (CSP) headers that restrict SW registration, or with Subresource Integrity (SRI) checks on served JavaScript files.

IX. CONCLUSION

We replicated key experiments from the IronSpider web-based PLC malware study, confirming the paper's firmware

growth analysis, the full six-step SW resurrection cycle including hardware replacement survival, and the three core attack capabilities on an alternative PLC platform (OpenPLC). We clarified the "24-hour cache limit" as the SW script update check interval, not a CacheStorage eviction policy, providing a more precise characterization of the persistence window.

To address the paper's stated detection gap, we implemented a three-tier behavioral detector. Tier 1 achieves zero false positives by correlating two browser-generated headers (`Sec-Fetch-Dest: empty` and a `.js`-suffixed `Referer`) to identify SW polling loops. Two of three tested evasion techniques defeat Tier 1, but both have straightforward countermeasures: extending the detection window and treating absent Referer headers as suspicious.

Future work includes cross-browser validation of the Fetch Metadata signals, integration with CSP-based SW registration restrictions, and multi-day false positive evaluation in a production-representative ICS environment.

REFERENCES

[1] D. Pickren, T. Shekari, S. Zonouz, and R. Beyah, "Compromising Industrial Processes using Web-Based PLC Malware," in *Proc. NDSS*, 2024. DOI: 10.14722/ndss.2024.23049.

[2] N. Falliere, L. O. Murchu, and E. Chien, "W32.Stuxnet Dossier," Symantec Security Response, Feb. 2011.

[3] L. Garcia, F. Brasser, M. H. Cintuglu, A.-R. Sadeghi, O. Mohammed, and S. A. Zonouz, "Hey, My Malware Knows Physics! Attacking PLCs with Physical Model Aware Rootkit," in *Proc. NDSS*, 2017.

[4] A. Russell et al., "Service Workers 1," W3C Recommendation, Nov. 2022. https://www.w3.org/TR/service-workers/

[5] T. Alves and J. Morris, "OpenPLC: An Open Source Alternative to Automation," in *Proc. IEEE SmartComp*, 2014.

[6] L. Canali, M. Cova, G. Vigna, and C. Kruegel, "Prophiler: A Fast Filter for the Large-Scale Detection of Malicious Web Pages," in *Proc. WWW*, 2011.

[7] C. Curtsinger, B. Livshits, B. Zorn, and C. Seifert, "ZOZZLE: Fast and Precise In-Browser JavaScript Malware Detection," in *Proc. USENIX Security*, 2011.

[8] S. Fass, M. Backes, and B. Stock, "JaSt: Fully Syntactic Detection of Malicious (Obfuscated) JavaScript," in *Proc. DIMVA*, 2018.

[9] S. Fass, M. Backes, and B. Stock, "JStap: A Static Pre-Filter for Malicious JavaScript Detection," in *Proc. ACSAC*, 2019.